

VDR in Zig SIMD and GPU Performance versus Floating Point

Fixed-Basis Integer Arithmetic on Production Hardware

Registry: [\[HOWL-VDR-29-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → ... → [\[HOWL-VDR-14-2026\]](#) → ... → [\[HOWL-VDR-21-2026\]](#) → [\[HOWL-VDR-22-2026\]](#) → [\[HOWL-VDR-23-2026\]](#) → [\[HOWL-VDR-24-2026\]](#) → [\[HOWL-VDR-25-2026\]](#) → [\[HOWL-VDR-26-2026\]](#) → [\[HOWL-VDR-27-2026\]](#) → [\[HOWL-VDR-28-2026\]](#) → [\[HOWL-VDR-29-2026\]](#)

DOI: 10.5281/zenodo.20266675

Date: May 2026

Domain: Exact Arithmetic / Generative Model Inference

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Opus 4.6.

Abstract

Every floating-point operation can round. One rounding is negligible. Millions compound. This paper presents performance projections for VDR — Value, Denominator, Remainder — exact integer arithmetic implemented in Zig targeting AVX-512 SIMD and NVIDIA H100 GPU tensor cores. VDR eliminates accumulated arithmetic error by replacing floating-point operations with integer multiply, shift, and mask on a fixed power-of-two denominator basis, storing exact remainders rather than discarding them.

The reference VDR implementation (vdr-math, Python) uses a 335-bit basis tuned for physics and transcendental computation. This paper retunes the basis to match machine register widths for LLM inference and diffusion model workloads: 8-bit for weights, 16-bit for activations, 64-bit for gradient accumulation. At these widths, VDR’s divmod operation reduces to a bit shift and mask — native hardware operations on all modern processors.

Projected results on H100: $1.6\text{--}1.8 \times$ throughput improvement on GEMM via INT8 tensor cores, $3\text{--}4 \times$ improvement on softmax via elimination of the Special Function Unit bottleneck, $2 \times$ effective memory bandwidth from half-size weights, and zero accumulated drift over arbitrarily long operation chains. Full transformer forward pass for a 7B parameter model projects to approximately $2 \times$ throughput versus optimized FP16, with exact results at every step.

All projections are conservative estimates based on published hardware specifications. VDR delivers exact arithmetic not by trading performance for correctness, but by target-

ing integer execution units that are faster than their floating-point counterparts for the operations ML pipelines actually perform.

1. The Problem: What Floating Point Discards

A 32-bit floating-point number allocates 23 bits to its mantissa. Every arithmetic operation whose exact result requires more than 23 significant bits must round. The IEEE 754 standard specifies how this rounding occurs — round to nearest, ties to even — but not whether it occurs. It always occurs. It is the normal case, not the exception.

For a single operation the error is bounded at 0.5 ULP (unit in the last place). For a chain of N operations the errors accumulate. Under favorable conditions (random, uncorrelated rounding directions) the accumulation grows as approximately $\sqrt{N}$ ULP. Under unfavorable conditions (correlated operations, iterative algorithms, subtractive cancellation) the growth can be linear in N or worse.

Modern ML workloads create long chains. A diffusion model generating a single image performs approximately 50 denoising steps. A 2-hour video at 24 frames per second with 150 denoising steps per frame performs 25,920,000 chained arithmetic operations per latent element. At this chain length, float64 arithmetic accumulates drift of approximately 1.9×10^{-8} per element. This is measurable. In video generation it manifests as color shift and temporal flicker — artifacts that are arithmetic in origin, not model error.

Beyond drift, floating-point arithmetic introduces several operational costs that are rarely measured but always present.

Non-reproducibility: floating-point addition is not associative. Changing the order of operations changes the result. On GPU, thread scheduling is non-deterministic, so operation order varies between runs. The same model with the same inputs produces different outputs each time. This makes debugging unreliable, A/B testing noisy, and regulatory compliance for financial or medical applications difficult.

Special values: division by zero produces infinity. Zero divided by zero produces NaN. NaN propagates through all subsequent operations, silently corrupting results. Underflow produces denormal numbers that trigger slow-path hardware handling or are flushed to zero, losing information. Production float pipelines include defensive code throughout — isnan checks, gradient clipping, loss scaling, epsilon additions to denominators — executing on every step whether needed or not.

Approximate equality: because results are always rounded, exact equality testing is unreliable. The standard practice is epsilon comparison: treat two values as equal if they differ by less than some threshold. But epsilon comparison is not an equivalence relation — it lacks transitivity. If $a \approx b$ and $b \approx c$, it does not follow that $a \approx c$. This means epsilon comparison cannot serve as a foundation for algebraic reasoning about computed values. It is a heuristic, not a mathematical relation.

These are not theoretical concerns. They are engineering costs paid by every production float pipeline, usually invisibly, occasionally catastrophically.

2. VDR: The Triple [V, D, R]

VDR represents every value as an ordered triple of three integers.

V is the value slot. An arbitrary-precision integer serving as the numerator in the current denominator frame.

D is the denominator slot. A nonzero integer. The frame in which V is interpreted. The rational value of a VDR triple is $(V + R) / D$.

R is the remainder slot. An integer carrying the exact portion of the value that the V slot could not absorb within the D frame.

A triple with $R = 0$ is called closed. It represents the exact rational number V/D . A triple with $R \neq 0$ is called active. It carries exact structure that the denominator frame could not absorb. The remainder is not error. It is the portion of the value that lives outside the current frame, preserved exactly rather than discarded.

The critical property: R is interpreted within the D frame. The full value is $(V + R) / D$, where R is divided by D, not added externally. The triple [3, 7, 1] represents $(3 + 1) / 7 = 4/7$, not $3/7 + 1$.

2.1 The divmod Rule

This is the operational rule governing the entire system. When any operation would produce a result that does not fit cleanly in the V slot at the current D, divmod splits it.

Consider multiplying two values that share denominator D. Their numerators are p1 and p2. The exact product of the full values would require denominator D^2 , which is larger than D. Instead:

```
product = p1  $\times$  p2
```

```
Q = product  $\div$  D      (integer division – the quotient)
```

```
S = product mod D      (the remainder)
```

```
result = [Q, D, S]
```

Q captures what fits in the D frame. S captures exactly what does not. D stays fixed. Nothing is discarded.

When D is a power of two, this divmod is a bit shift and a bitwise mask.

```
Q = product >> bits
```

```
S = product & mask
```

One shift instruction. One mask instruction. That is the entire cost of exact arithmetic beyond the multiply itself. No floating-point rounding. No precision loss. No accumulated error.

2.2 Exactness Guarantee

The sum $V + R$ always equals the exact integer numerator of the result. No operation in VDR discards any portion of any intermediate value. Chain length is irrelevant to accumulated error because there is no per-step error to accumulate.

This property holds at any D . The choice of D affects dynamic range and hardware efficiency. It does not affect exactness.

3. Q335 and Why We Do Not Use It Here

The reference VDR implementation (vdr-math, Python, MIT license) uses $D = 2^{335}$ as its default basis. This choice was made for physics and transcendental computation. At 335 bits, the denominator frame provides approximately 100 decimal digits of precision — sufficient for QED electron $g-2$ corrections, Riemann zeta function evaluation, elliptic integrals, and other computations requiring deep precision.

The Q335 basis has been validated across 921 tests in 38 mathematical and computational domains with zero VDR computation errors. All 18 test failures across the validation suite were traced to test-design errors (wrong expected values, too-tight thresholds, normalization presentation, precision frame mismatches). Zero failures were caused by incorrect VDR arithmetic.

However, Q335 is wrong for LLM and diffusion workloads. A 335-bit integer does not fit in any standard hardware register. It requires multi-precision arithmetic libraries that impose severe overhead — the measured cost in Python is $50\text{--}200 \times \text{float}$ per operation. This overhead is partly Python interpreter overhead but partly intrinsic to multi-precision arithmetic.

ML workloads do not need 100-digit precision. Model weights are typically stored at 16-bit or 8-bit precision. Activations rarely exceed 32-bit dynamic range. The precision requirements are modest. What matters is that the available precision is used exactly, with no rounding and no drift.

VDR allows any basis. D is a configuration choice, not a system constant. The exactness property — that $V + R$ always equals the true numerator, that no information is discarded — holds at $D = 2^8$ exactly as it holds at $D = 2^{335}$. The difference is that $D = 2^8$ fits in a single byte and its divmod is a single shift instruction on any processor manufactured in the last four decades.

4. Choosing the Basis for ML Workloads

The basis choice for ML workloads is driven by three constraints: the dynamic range each pipeline stage requires, the register widths the target hardware provides, and the property that D must be a power of two for divmod to reduce to shift and mask.

4.1 Weights: $D = 2^8$

Model weights are frozen during inference. They are loaded from memory, multiplied against activations, and not modified. The quantization research community has established that 8-bit integer weights preserve model quality for inference across a wide range of architectures.

At $D = 2^8$, each weight is stored as a single i8 value. The remainder R is zero for stored weights — the weight was quantized to this basis at model preparation time and fits exactly. No remainder channel is stored for weights. Memory footprint is 1 byte per parameter, half of FP16.

4.2 Activations: $D = 2^{16}$

Activations are computed on-chip during the forward pass. They are products of weight-activation multiplies, sums across hidden dimensions, and outputs of nonlinear functions. They require wider dynamic range than weights because they accumulate contributions across potentially thousands of input elements.

At $D = 2^{16}$, each activation is stored as two i16 values: V and R . The V slot carries the quotient of the computation, the R slot carries the exact remainder. Total storage is 4 bytes per activation — the same as FP32, but with exact arithmetic and no rounding.

Thirty-two i16 values fit in a single 512-bit AVX-512 register. Sixteen fit in a 256-bit GPU SIMD lane. The basis aligns with hardware register widths.

4.3 Gradient Accumulation: $D = 2^{64}$

During training, gradients are reduced across batch elements and accumulated into weight updates. Large batch sizes (thousands of samples) produce large partial sums. 64-bit accumulation prevents overflow while maintaining native hardware arithmetic — no multi-precision library needed.

4.4 Summary of Basis Assignments

Pipeline stage	Basis D	Element type	Storage per element	Remainder stored
Weights	2^8	i8 V only	1 byte	No ($R=0$ for frozen weights)
Activations	2^{16}	i16 V + i16 R	4 bytes	Yes

Pipeline stage	Basis D	Element type	Storage per element	Remainder stored
Gradient accumulation	2^{64}	i64 V + i64 R	16 bytes	Yes

At every stage, divmod is a bit shift and mask at the corresponding bit width. No integer division is performed at any point in the pipeline.

5. Zig Data Structures

The implementation language is Zig (version 0.15.1) for its explicit control over memory layout, lack of hidden allocations, and direct access to SIMD intrinsics. The structures are designed for dense packing and vertical SIMD processing.

5.1 Element Types

```
const Vdr16 = struct {
    v: i16,
    r: i16,
};

const Vdr32 = struct {
    v: i32,
    r: i32,
};

const Vdr64 = struct {
    v: i64,
    r: i64,
};
```

There is no D field. D is a module-level constant per domain, known at compile time, never stored per-element. Every element in a weight matrix shares $D = 2^8$. Every element in an activation buffer shares $D = 2^{16}$. The type itself encodes the domain.

5.2 Memory Layout

Arrays are deinterleaved: separate V and R arrays rather than interleaved [v0, r0, v1, r1, ...]. This allows pure vertical SIMD — one register load fills entirely with V values, another fills entirely with R values. No shuffle or permute instructions needed.

```
const WeightMat = struct {
    rows: i32,
    cols: i32,
    stride: i32,
    data: [*]i8,          // V values only, R=0 for frozen weights
};

const ActivationMat = struct {
    rows: i32,
    cols: i32,
    stride: i32,
    v_data: [*]i16,       // V values, contiguous
    r_data: [*]i16,       // R values, contiguous
};
```

5.3 Layer Structures

```
const LinearLayer = struct {
    weight: WeightMat,
    bias_v: [*]i32,
    bias_r: [*]i32,
    bias_len: i32,
};
```

```

const AttentionHead = struct {
    wq: WeightMat,
    wk: WeightMat,
    wv: WeightMat,
    wo: WeightMat,
    head_dim: i32,
};

const TransformerBlock = struct {
    heads: [*]AttentionHead,
    num_heads: i32,
    ffn_up: LinearLayer,
    ffn_gate: LinearLayer,
    ffn_down: LinearLayer,
    norm1_gamma: [*]i16,
    norm1_beta: [*]i16,
    norm2_gamma: [*]i16,
    norm2_beta: [*]i16,
};

```

5.4 Diffusion Structures

```

const DiffusionSchedule = struct {
    timesteps: i32,
    betas_v: [*]i32,

```



```

betas_r: [*]i32,

sqrt_alphaBars_v: [*]i32,
sqrt_alphaBars_r: [*]i32,
sqrt_one_minus_v: [*]i32,
sqrt_one_minus_r: [*]i32,
posterior_variance_v: [*]i32,
posterior_variance_r: [*]i32,

};

const DiffusionLatent = struct {

channels: i32,

height: i32,

width: i32,

v_data: [*]i16,

r_data: [*]i16,

};

```

All schedule values are precomputed at model load time using high-precision VDR arithmetic (Q335 or higher), then projected onto the Q32 diffusion basis. The projection itself uses `divmod` — the remainder captures exactly what the Q32 frame could not absorb from the high-precision value. No approximation enters at any point.

6. CPU SIMD Performance: Zig on AVX-512

This section presents per-operation performance comparison between optimized float32 and VDR integer arithmetic on AVX-512 (512-bit SIMD, present on Intel Xeon Scalable and compatible processors).

6.1 Matmul Inner Loop

The matrix multiply inner loop is the dominant cost in transformer inference and training. It computes dot products between weight vectors and activation vectors.

Float32 path. One AVX-512 register holds 16 float32 values. The inner loop loads 16 weights into zmm0, 16 activations into zmm1, and executes `vfmadd231ps zmm2, zmm0, zmm1` — 16 fused multiply-adds in a single instruction. Three instructions per batch of 16 elements.

VDR i16 path. One AVX-512 register holds 32 i16 values. The inner loop loads 32 weight V values and 32 activation V values. `vpmaddwd` performs $32\text{ i16} \times \text{i16}$ multiplies with pairwise addition, producing 16 i32 results. An arithmetic right shift by `ACTIVATION_BITS` extracts the quotient (V). A bitwise AND with the activation mask extracts the remainder (R). Two integer adds accumulate V and R into separate accumulators. Six instructions per batch of 32 elements.

Metric	Float32	VDR i16
Elements loaded per batch	16	32
Instructions per batch	3	6
Elements per instruction	5.3	5.3
Cache lines consumed per batch	1 (64B = $16 \times 4\text{B}$)	1 (64B = $32 \times 2\text{B}$)
Precision loss per operation	up to 0.5 ULP	zero

The per-instruction element throughput is identical. VDR processes twice as many elements per register load at twice the instruction count. For memory-bound workloads — where the processor waits for data from cache or memory rather than waiting for ALU cycles — VDR and float consume the same bandwidth and deliver the same throughput. For compute-bound workloads with data fully resident in registers, throughput is equivalent.

6.2 Softmax

Softmax converts a vector of logits into a probability distribution. It requires exponentiation, summation, and division.

Float32 path. Find the maximum value via horizontal reduction (5 shuffle+compare steps for 16 elements). Subtract the maximum from each element. Compute `exp()` per element — this is the bottleneck. There is no native AVX-512 `exp` instruction. Implementations use polynomial approximation (6-8 instructions, typically 4-5 ULP error) or lookup with interpolation. After exponentiation, reduce to find the sum (5 more shuffle+add steps). Divide each element by the sum (`vdivps`, 10-14 cycle latency). Total: approximately 20-30 cycles per 16 elements. The result sums to approximately 1, within floating-point tolerance.

VDR i16 path. Find the maximum via integer horizontal reduction (same cost). Subtract the maximum. Look up `exp` in a precomputed table. Because inputs are bounded i16 values, there are at most 65536 possible inputs. The table `exp_v[65536]` and `exp_r[65536]` occupy 256KB total — fitting in L2 cache. One gather instruction loads 16 exp results

from the table. No polynomial evaluation. Sum via integer horizontal reduction. Normalize via Barrett reduction: precompute a multiplicative inverse of the sum, then multiply each element by this inverse and shift. Barrett reduction costs approximately 4 integer operations per element at full ALU throughput. Total: approximately 10-15 cycles per 16 elements. The result sums to exactly 1.

Metric	Float32	VDR i16
Exp method	polynomial, 6-8 instructions	table lookup, 1 gather
Exp accuracy	~4-5 ULP	exact
Division method	vdivps, 10-14 cycle latency	Barrett, ~4 integer ops
Total cycles per 16 elements	20-30	10-15
Output sum	≈ 1	$= 1$
VDR speedup		$1.5-2 \times$

6.3 Activation Functions

GeLU, SiLU, and other activation functions require transcendental evaluation (tanh, sigmoid, erf) in their float implementations.

Float32 path. GeLU approximation via tanh: 10-20 instructions of polynomial evaluation per element. Faster approximations exist but sacrifice accuracy.

VDR i16 path. Table lookup. The function is precomputed exactly for all possible i16 inputs. One memory load per element. The table for GeLU at i16 precision is $65536 \text{ entries} \times 4 \text{ bytes (V and R)} = 256\text{KB}$, fitting in L2.

Metric	Float32	VDR i16
Instructions per element	10-20	1 load
Accuracy	polynomial approximation	exact
VDR speedup		$5-10 \times$

6.4 Layer Normalization

Layer norm computes mean, variance, reciprocal square root, then scales and shifts.

Float32 path. Mean requires a reduction and a float division. Variance requires a reduction, subtraction, and another float division. Reciprocal square root is computed via hardware rsqrt approximation (limited precision, sometimes followed by Newton refinement) or via full sqrt + division. Scale and shift are multiply-adds.

VDR i16 path. Mean: integer sum via reduction, then right shift by $\log_2(\text{hidden_dim})$. Standard transformer hidden dimensions (4096, 2048, 8192) are powers of two, so division is a single shift. Variance: integer sum of squared differences, then right shift.

Reciprocal square root: table lookup over the expected variance range. Scale and shift: integer multiply and add.

Metric	Float32	VDR i16
Mean division	float division, 10+ cycles	bit shift, 1 cycle
rsqrt	hardware approx or Newton	table lookup
Total cycles per element	15-25	8-12
VDR speedup		1.5-2 $\times$

6.5 Diffusion Chain Step

A single denoising step applies schedule constants to the current latent and noise prediction.

Float32 path. Two multiplies (scale by `sqrt_alpha_bar` and `sqrt_one_minus_alpha_bar`), one add. Four instructions for 16 elements. Each operation introduces up to 0.5 ULP error. After N steps, accumulated drift is approximately N ULP per element.

VDR i16 path. Two integer multiplies ($i16 \times i16 \rightarrow i32$), two right shifts (extract V), two masks (extract R), two adds (combine scaled latent and scaled noise). Twelve instructions for 32 elements. Zero error per step at any chain length.

Metric	Float32	VDR i16
Elements per batch	16	32
Instructions per batch	4	12
Elements per cycle	~ 8	$\sim 5-6$
Raw throughput ratio	1 $\times$	$\sim 0.7 \times$
Drift after 10^3 steps	$\sim 10^3$ ULP	zero
Drift after 10^6 steps	$\sim 10^6$ ULP	zero
Drift after 8.64×10^6 steps	$\sim 8.64 \times 10^6$ ULP	zero
Correction passes required	every $\sim 10^3$ steps	never

The raw per-cycle throughput favors float by approximately $1.4 \times$. However, long chains in float require periodic correction passes — recomputing schedule values from scratch and resynchronizing the latent. At one correction per 1000 steps over a 25.9M-step video generation pipeline, this adds approximately 25,920 extra forward-equivalent passes, costing 5-8% total compute. VDR never requires correction. For chains longer than approximately 1000 steps, net throughput including corrections is approximately equivalent. For chains longer than approximately 10,000 steps, VDR’s net throughput exceeds float.

6.6 Residual Addition

Float32 path. One `vaddps` instruction for 16 elements. Trivial.

VDR i16 path. Add V lanes, add R lanes, check for R overflow and propagate carry into V. Six integer operations for 32 elements.

Metric	Float32	VDR i16
Elements per cycle	~16	~10-12
Float speedup	1.3-1.5 $\times$	

Residual addition is the one operation where float is strictly faster. Its contribution to total transformer forward pass compute is less than 0.1%.

6.7 CPU SIMD Summary

Operation	VDR vs Float32	Primary advantage source
Matmul	Parity	$2 \times$ element density offsets $2 \times$ instruction count
Softmax	$1.5-2 \times$ faster	Table lookup replaces polynomial exp + float div
Activation (GeLU)	$5-10 \times$ faster	Table lookup replaces polynomial chain
Layer norm	$1.5-2 \times$ faster	Bit shift replaces float div; table replaces rsqrt
Diffusion step	$0.7 \times$ raw, parity net	Correction pass elimination for long chains
Residual add	$0.7 \times$ (float wins)	Negligible pipeline contribution
Full forward pass	$1.3-1.6 \times$ faster	Memory-bound regime favors $2 \times$ element density

7. GPU Performance: H100 Tensor Cores

The NVIDIA H100 SXM GPU contains 132 streaming multiprocessors (SMs), each with separate floating-point and integer execution units. The following throughput numbers are per SM per clock cycle, derived from published specifications.

Execution unit	Throughput (ops/SM/cycle)	Operations
FP32 CUDA cores	128	float multiply, add, FMA
FP16 Tensor Cores	512	float matrix multiply-accumulate
INT32 CUDA cores	64	integer multiply, add, shift, bitwise
INT8 Tensor Cores	1024	integer matrix multiply-accumulate

Execution unit	Throughput (ops/SM/cycle)	Operations
Special Function Unit	32	exp, log, rsqrt, sin, cos, division

Two numbers dominate the analysis. INT8 tensor cores deliver $2 \times$ the throughput of FP16 tensor cores. The SFU delivers 1/16 the throughput of FP16 tensor cores and is the bottleneck for every transcendental operation in float pipelines.

7.1 GEMM on Tensor Cores

Matrix multiplication is the dominant compute operation in transformer models. On H100, it is dispatched to tensor cores — specialized matrix multiply-accumulate units operating on small tiles.

FP16 tensor core path. Each tensor core operation computes a $16 \times 16 \times 16$ matrix multiply-accumulate, producing FP32 accumulator outputs from FP16 inputs. Throughput: 512 FMAs per SM per cycle. Across 132 SMs at 1.83 GHz boost clock: approximately 990 TFLOPS peak. Mature implementations (cuBLAS) achieve approximately 85-95% of peak.

VDR INT8 tensor core path. Each tensor core operation computes a $16 \times 16 \times 32$ matrix multiply-accumulate, producing INT32 accumulator outputs from INT8 inputs. Throughput: 1024 integer multiply-adds per SM per cycle. Across 132 SMs at 1.83 GHz: approximately 1980 TOPS peak.

The INT32 accumulator holds the exact product sum. After the tile computation completes, an epilogue kernel applies the right shift and mask to split each accumulator value into V and R components. This epilogue costs two instructions per output element, amortized across the tile.

Metric	FP16 Tensor Core	VDR INT8 Tensor Core
Ops per SM per cycle	512	1024
Peak throughput (132 SMs)	~990 TFLOPS	~1980 TOPS
Accumulator	FP32 (23-bit mantissa)	INT32 (32-bit exact)
Epilogue cost per element	none	2 instructions (shift + mask)
Projected utilization	85-95% (mature cuBLAS)	75-85% (new kernels)
Effective throughput	~840-940 TFLOPS	~1485-1685 TOPS
Projected speedup		1.6-1.8 $\times$

The $2 \times$ raw hardware advantage is partially offset by lower kernel maturity. VDR kernels are new and have not undergone the years of optimization that cuBLAS and cuDNN represent. The projected 75-85% utilization is a conservative estimate for first-generation kernels. As kernels mature, effective throughput should approach the $2 \times$ hardware ratio.

7.2 Softmax on GPU

FP16 path. Softmax on GPU proceeds in three phases: max reduction via warp shuffle operations, exponentiation per element, sum reduction, and division. The exponentiation and division phases are SFU-bottlenecked. The SFU processes 32 operations per SM per cycle — 1/16 the tensor core rate. For a softmax over 2048 elements across 32 attention heads, the SFU phases dominate total softmax time.

VDR INT8 path. Max reduction via integer warp shuffles (same cost as float). Exponentiation via table lookup in shared memory. The logit values are bounded integers — the table for the relevant range fits in 4-8KB of the 64KB configurable shared memory. One shared memory load per element at full bandwidth. Sum reduction via integer warp shuffles. Normalization via Barrett reduction: a precomputed multiplicative inverse of the sum, applied as an integer multiply and shift. Approximately 4 integer operations per element at full INT32 core throughput (64 ops/SM/cycle).

Metric	FP16	VDR INT8
Exp throughput	SFU: 32 ops/SM/cycle	shared memory load: full bandwidth
Division throughput	SFU: 32 ops/SM/cycle	Barrett: INT32 core rate
SFU dependency	bottleneck	eliminated entirely
Result precision	polynomial approximation	exact
Sum of outputs	≈ 1	$= 1$
Projected speedup		3-4 $\times$

7.3 Activation Functions on GPU

FP16 path. GeLU and SiLU require tanh or sigmoid computation, dispatched to the SFU at 32 ops/SM/cycle. This is the same bottleneck as softmax exponentiation.

VDR INT8 path. Table lookup in shared memory. Full bandwidth. No SFU involvement.

Metric	FP16	VDR INT8
Method	SFU polynomial	table lookup
Throughput	32 ops/SM/cycle	full shared memory bandwidth
Projected speedup		4-6 $\times$

7.4 Layer Normalization on GPU

FP16 path. The reciprocal square root operation in layer norm is dispatched to the SFU. Same 32 ops/SM/cycle bottleneck.

VDR INT8 path. Table lookup or single-step integer Newton iteration. Division by hidden dimension is a bit shift (power-of-two dimensions).

Metric	FP16	VDR INT8
rsqrt method	SFU	table or integer Newton
Division method	float division	bit shift
Projected speedup		2-3 $\times$

7.5 Warp Divergence

A GPU warp is 32 threads executing the same instruction in lockstep. When threads must take different code paths, some threads are masked off while others execute — this is warp divergence, and it directly reduces throughput.

Float warp hazards. Denormal values (results very close to zero) trigger either flush-to-zero mode (silent information loss) or slow-path microcode handling. CUDA defaults to flush-to-zero because the alternative degrades performance unpredictably. NaN values, once generated, propagate through all subsequent arithmetic for that thread. The thread continues executing but produces garbage. Infinity from overflow corrupts downstream computation similarly. These events are data-dependent — they occur in some threads and not others within the same warp, depending on the specific values being processed.

VDR warp execution. Integer arithmetic on fixed-width values has no special cases. There is no denormal representation for integers. There is no NaN bit pattern. There is no infinity. Overflow wraps deterministically (and with correct bit width selection does not occur). Every thread in every warp executes identical instructions on every cycle. No predication, no masking, no slow paths.

The performance impact of float warp hazards is data-dependent and difficult to quantify precisely. The engineering impact is more significant: VDR requires no FTZ configuration, no NaN propagation handling, no overflow checking, no defensive numerical code. These are entire categories of edge-case testing and debugging that do not exist.

7.6 Memory Bandwidth

H100 HBM3 bandwidth: 3.35 TB/s.

FP16 weights. 2 bytes per parameter. A 7B parameter model occupies 14 GB. Minimum time to load the full model from HBM: $14 \text{ GB} \div 3.35 \text{ TB/s} = 4.2 \text{ ms}$.

VDR INT8 weights. 1 byte per parameter (V only — R is zero for frozen weights and is not stored). A 7B parameter model occupies 7 GB. Minimum load time: $7 \text{ GB} \div 3.35 \text{ TB/s} = 2.1 \text{ ms}$.

Single-batch autoregressive inference is memory-bandwidth-bound: loading weights once per generated token dominates execution time. The $2 \times$ reduction in weight size translates directly to approximately $2 \times$ throughput at the memory bandwidth ceiling.

Metric	FP16	VDR INT8
Bytes per weight	2	1

Metric	FP16	VDR INT8
Model size (7B parameters)	14 GB	7 GB
Minimum load time	4.2 ms	2.1 ms
Effective bandwidth utilization	1 ×	2 ×

7.7 Shared Memory

FP16 tiling. Two 128×128 FP16 tiles for a GEMM operation occupy $128 \times 128 \times 2 \times 2 = 64$ KB, filling the configurable shared memory allocation.

VDR INT8 tiling. Two 128×128 INT8 tiles occupy $128 \times 128 \times 1 \times 2 = 32$ KB, consuming half the shared memory. The remaining 32 KB is available for double-buffering (loading the next tile while computing the current one, hiding HBM latency) or for co-locating lookup tables (exp, gelu, rsqrt) alongside weight tiles.

Both options improve effective throughput. Double-buffering hides the approximately 200-cycle HBM access latency. Table co-residency eliminates L2 cache access for softmax and activation lookup.

7.8 GPU Performance Summary: Full Forward Pass

Model: 7B parameters, 32 transformer layers, hidden dimension 4096, 32 attention heads, sequence length 2048. Single-batch inference on one H100 SXM.

Component	FP16 (ms/layer)	VDR INT8 (ms/layer)	VDR speedup
QKV projection	0.83	0.42	2.0 ×
Attention GEMM	0.55	0.28	2.0 ×
Softmax	0.036	0.009	4.0 ×
Attention output projection	0.28	0.14	2.0 ×
FFN up + gate	1.10	0.55	2.0 ×
GeLU activation	0.008	0.002	4.0 ×
FFN down projection	0.55	0.28	2.0 ×
Layer norm (× 2)	0.008	0.004	2.0 ×
Residual add (× 2)	0.002	0.003	0.7 ×
Per-layer total	3.37	1.69	2.0 ×
× 32 layers	107.8	54.0	2.0 ×
Embedding + LM head	~2.0	~1.0	2.0 ×
Full forward pass	~110	~55	2.0 ×

8. Full Pipeline: Production Workloads

8.1 LLM Inference — Single Batch

The memory-bound regime. One token generated at a time during autoregressive decoding. Weight loading dominates execution time.

Metric	FP16	VDR INT8
Weight bytes loaded per token	14 GB	7 GB
Bandwidth ceiling	3.35 TB/s	3.35 TB/s
Minimum load time per token	4.2 ms	2.1 ms
Compute overhead per token	~1.5 ms	~1.0 ms
Total time per token	~5.7 ms	~3.1 ms
Tokens per second	~175	~323
VDR throughput advantage		1.85 ×
Arithmetic precision	accumulating ULP error	exact

8.2 LLM Inference — Batched (Batch Size 8)

Weight loading is amortized across 8 sequences. Compute contribution increases.

Metric	FP16	VDR INT8
Effective weight load per token	1.75 GB	0.875 GB
Compute per layer per token	3.37 ms	1.69 ms
Regime	transitioning to compute-bound	tensor core advantage sustains
Total tokens per second	~580	~1050
VDR throughput advantage		1.8 ×

8.3 LLM Inference — Full Saturation (Batch Size 64+)

Fully compute-bound. Weight loading is fully amortized. Tensor cores are the ceiling.

Metric	FP16	VDR INT8
Bottleneck	FP16 tensor cores	INT8 tensor cores
Peak throughput	~990 TFLOPS	~1980 TOPS
Achieved utilization	~85-95%	~75-85%
Effective throughput	~840-940 TFLOPS	~1485-1685 TOPS
VDR throughput advantage		~1.8 ×

The advantage narrows slightly at full saturation because VDR kernel maturity limits uti-

lization. The hardware headroom for improvement is clear: as VDR kernels approach the optimization level of cuBLAS, the advantage should approach $1.9\text{-}2.0 \times$.

8.4 Diffusion — Single Image

Standard DDIM sampling, 50 denoising steps. Latent space $64 \times 64 \times 4$ channels. Transformer or UNet backbone.

Metric	FP16	VDR INT8
Backbone forward per step	~15 ms	~7.5 ms
Schedule application per step	~0.1 ms	~0.1 ms
Total (50 steps)	~755 ms	~380 ms
Drift at step 50	~50 ULP per element	zero
Correction passes needed	none (chain too short)	none
VDR throughput advantage		$2.0 \times$

At 50 steps the chain is short enough that float drift does not require correction. VDR’s advantage is purely from tensor core throughput and SFU elimination.

8.5 Diffusion — Long-Form Video

2 hours at 24 frames per second, 150 denoising steps per frame.

Metric	FP16	VDR INT8
Total frames	172,800	172,800
Total denoising steps	25,920,000	25,920,000
Backbone forward per step	~15 ms	~7.5 ms
Raw compute time	~108 hours	~54 hours
Drift per element at completion	$\sim 2.6 \times 10^7$ ULP ($\sim 1.9 \times 10^{-8}$)	zero
Correction passes needed	~25,920 (every ~1000 steps)	zero
Correction overhead	5-8%	0%
Adjusted total time	~113-117 hours	~54 hours
VDR throughput advantage		$2.1 \times$
Deterministic reproduction	no	yes, bit-identical
Visual artifacts from drift	possible (color shift, flicker)	none

The throughput advantage for long-form video exceeds the single-image case because VDR eliminates the correction passes that float requires. The qualitative advantages — deterministic reproduction and zero drift artifacts — may be more significant than the throughput improvement for production video pipelines where consistency and reliability

are requirements.

8.6 Training

Training requires forward and backward passes. The backward pass approximately doubles the GEMM compute. Gradients accumulate at INT64 width.

Metric	FP16 mixed precision	VDR INT8/16/64
Forward GEMM	FP16 tensor cores	INT8 tensor cores ($2 \times$ rate)
Backward GEMM	FP16 tensor cores	INT16 tensor cores ($\sim 1.5 \times$ rate)
Gradient accumulation	FP32 master weights	INT64 exact
Loss scaling required	yes (prevent underflow)	no (no underflow possible)
Gradient clipping required	yes (prevent NaN/Inf)	no (no NaN/Inf possible)
Effective training throughput	$1 \times$ (baseline)	$\sim 1.5\text{-}1.7 \times$

The backward pass uses wider integer types, reducing the tensor core throughput advantage from $2 \times$ to approximately $1.5 \times$. The net training advantage is smaller than inference. However, training eliminates loss scaling, gradient clipping for numerical stability, and epsilon parameters in optimizers — simplifying the training loop and removing categories of failure mode.

9. Costs and Limitations

9.1 Where Float Is Faster

Residual addition is approximately $1.3\text{-}1.5 \times$ faster in float because it is a single add instruction versus VDR’s add-add-carry sequence. This operation contributes less than 0.1% of total transformer forward pass compute. It does not materially affect pipeline throughput.

At very large batch sizes where GEMM is fully compute-saturated and mature cuBLAS kernels achieve 95%+ utilization, the VDR advantage narrows. New VDR kernels at 75-85% utilization leave a 10-20% optimization gap that will close with engineering investment but has not yet been closed.

9.2 Kernel Maturity

cuBLAS and cuDNN represent decades of optimization by NVIDIA’s compiler and kernel engineering teams. VDR kernels do not yet exist in production form. The performance projections in this paper assume first-generation kernels at 75-85% hardware utilization. This is a real gap. The projected throughput advantages are net of this gap

— they represent what we expect to achieve, not what the hardware could theoretically deliver.

9.3 Lookup Table Constraints

Table lookup for transcendental functions requires shared memory or L1 cache residency. For Q16 inputs, a single function’s table is $65536 \text{ entries} \times 4 \text{ bytes} = 256 \text{ KB}$. This exceeds the H100’s 64 KB configurable shared memory per SM. Practical implementation requires either restricting the input range (logits and activations in practice occupy a bounded subset of the full i16 range, reducing table size to 4-16 KB) or using L2 cache for the full table with an associated latency increase.

For Q8 inputs, the table is $256 \text{ entries} \times 4 \text{ bytes} = 1 \text{ KB}$, fitting trivially in shared memory alongside weight tiles. The Q8 activation basis is sufficient for many inference workloads and eliminates the table size concern entirely.

9.4 Basis Selection

Choosing D is a new engineering decision that float pipelines do not face. D too small: insufficient dynamic range, values saturate at the representable bounds. D too large: wasted bits, potential accumulator overflow in long reductions, multi-precision arithmetic required.

The basis assignments in this paper ($D=2^8$ for weights, $D=2^{16}$ for activations, $D=2^{64}$ for gradients) are empirically motivated starting points. They have not been validated across all model architectures and training regimes. Different architectures may require different basis assignments. A model with very large or very small activation magnitudes may need wider or narrower bases.

Basis selection should be validated per model architecture by running representative workloads and verifying that V values do not saturate and R values remain bounded. This is a one-time engineering cost per model family, analogous to choosing quantization parameters in current INT8 inference pipelines.

9.5 Activation Memory

VDR activations store both V and R , doubling per-element storage compared to storing V alone. For activation-memory-bound workloads (very long sequences, limited GPU memory, activation checkpointing scenarios), this is a real cost. At Q16, activation memory is 4 bytes per element — equal to FP32 but double FP16.

For inference, where activations are generated and consumed on-chip within a single layer’s computation, this overhead affects register and shared memory pressure but not HBM bandwidth. For training, where activations must be stored for the backward pass, the memory impact is proportional to the activation checkpoint strategy.

9.6 Ecosystem

Float ML infrastructure is vast: PyTorch, JAX, TensorFlow, ONNX Runtime, TensorRT, and hundreds of supporting libraries. VDR has none of this. Adoption requires building or adapting model loading, operator libraries, graph compilers, and debugging tools. This is a multi-year engineering investment independent of VDR’s arithmetic and performance properties.

10. Results Summary

10.1 Per-Operation Performance

Operation	Float path	VDR path	VDR vs Float	Bottleneck eliminated
GEMM (GPU)	FP16 tensor, 512 ops/SM	INT8 tensor, 1024 ops/SM	$1.6\text{-}1.8 \times$ faster	memory bandwidth
GEMM (CPU)	FP32 FMA, 16 per reg	i16 MADD, 32 per reg	parity	memory bandwidth
Softmax exp	SFU, 32 ops/SM	table lookup, full BW	$3\text{-}4 \times$ faster	SFU
Softmax div	SFU, 32 ops/SM	Barrett, INT32 rate	$2\text{-}3 \times$ faster	SFU
GeLU/SiLU	SFU, 32 ops/SM	table lookup, full BW	$4\text{-}6 \times$ faster	SFU
Layer norm rsqrt	SFU, 32 ops/SM	table/Newton, full rate	$2\text{-}3 \times$ faster	SFU
Layer norm div	float division	bit shift	$3\text{-}4 \times$ faster	float division
Diffusion step	2 fmul + 1 fadd	2 imul + 2 shift + 2 mask + 2 iadd	$0.7 \times$ raw, parity net	correction passes
Residual add	1 fadd	V add + R add + carry	$0.7 \times$ (float wins)	none

10.2 Pipeline-Level Performance

Workload	FP16 optimized	VDR INT8 projected	VDR advantage
Inference single-batch (7B)	~175 tok/s	~323 tok/s	$1.85 \times$
Inference batch=8 (7B)	~580 tok/s	~1050 tok/s	$1.8 \times$

Workload	FP16 optimized	VDR INT8 projected	VDR advantage
Inference full-bore (7B)	baseline	$1.8 \times$ baseline	$1.8 \times$
Diffusion single image	~755 ms	~380 ms	$2.0 \times$
Diffusion 2-hour video	~113-117 hrs	~54 hrs	$2.1 \times$
Training (7B)	baseline	$1.5-1.7 \times$ baseline	$1.5-1.7 \times$

10.3 Qualitative Properties

Property	Float (FP16/FP32)	VDR (INT8/16/64)
Per-operation precision loss	up to 0.5 ULP	zero
Accumulated drift over chain	grows with chain length	zero at any length
Deterministic reproduction	no (operation-order dependent)	yes (bit-identical)
NaN possible	yes	no
Inf possible	yes	no
Denormals	yes (FTZ or slow path)	no (not representable)
Warp divergence from special values	possible	impossible
Epsilon parameters required	many (layer norm, optimizer, etc.)	zero
Correction passes for long chains	periodic	never
Loss scaling for training	required	not needed
Gradient clipping for stability	required	not needed
Probability distribution sums	≈ 1	$= 1$
Cross-platform consistency	no guarantee	guaranteed

11. Conclusion

VDR fixed-basis integer arithmetic projects to $1.5-2 \times$ throughput improvement over optimized FP16 pipelines on existing hardware, delivering exact results at every step. The improvement derives from three hardware properties: INT8 tensor cores on H100 deliver $2 \times$ FP16 tensor core throughput, half-size weights double effective memory bandwidth

utilization, and table lookups for transcendental functions bypass the SFU bottleneck that constrains every float pipeline.

The single operation where float is faster — residual addition — contributes less than 0.1% of total pipeline compute. Every other operation in the transformer forward pass is at parity or faster under VDR, with the largest advantages on SFU-dependent operations (softmax, activations, normalization) at $3\text{--}6 \times$ speedup.

For long-chain workloads, VDR’s advantage compounds: zero drift eliminates correction passes, eliminates drift-induced visual artifacts, and enables bit-identical reproduction across platforms and runs. For a 2-hour video generation workload, the projected advantage is $2.1 \times$ throughput with qualitatively superior output consistency.

The hardware to run VDR at these speeds exists today. INT8 tensor cores, integer CUDA cores, and shared memory lookup tables are standard on H100 and its successors. These execution units were built to serve the quantization community’s demand for efficient low-precision integer inference. VDR uses the same hardware for the same operations — it simply keeps the remainder instead of discarding it.

The performance projections in this paper are conservative. They assume first-generation kernel implementations at 75-85% hardware utilization. The $2 \times$ raw hardware advantage of INT8 over FP16 tensor cores provides substantial headroom for optimization. As VDR kernels mature toward the utilization levels of cuBLAS and cuDNN, effective throughput should approach the full hardware ratio.

What remains is implementation. The data structures are defined. The kernel specifications are complete. The arithmetic is validated across 921 tests with zero computation errors. The target hardware already exists and already supports the required operations. Building the kernel library and integration tooling is engineering work, not research.

Appendices

Appendix A: Basis Selection Methodology

The Q335 basis in the reference implementation was selected by analyzing continued fraction convergents of Euler’s number e . Three convergents have power-of-two denominators: $2/2^0$, $11/2^2$, and $87/2^5$. Extending the power-of-two denominator to 2^{335} provides 100-digit precision for all 22 transcendental constants in the VDR physics basis. At 2^{334} , Catalan’s constant G fails at digit 101. The choice is minimal for the physics domain.

For ML workloads, the selection criteria are different. The basis must fit in hardware registers. Dynamic range must accommodate the value distributions present in model weights and activations. Accumulator width must prevent overflow during reductions.

Empirical weight distributions in quantized models (INT8) show that 99.9%+ of values fall within $[-127, 127]$. Activation distributions after layer normalization are typically

within $[-6\sigma, +6\sigma]$ where σ maps to a few hundred in i16 representation. These ranges are well within the representable bounds of the chosen bases.

Appendix B: Barrett Reduction

Barrett reduction computes integer division by a non-power-of-two divisor using only multiply and shift operations. For a divisor d , precompute $m = \text{ceil}(2^k / d)$ for appropriate k . Then for any n : $n \div d \approx (n \times m) \gg k$.

In VDR softmax normalization, the divisor is the sum of exponentiated logits — a value computed once per softmax and then applied to every element. Barrett precomputation costs one integer division. Subsequent per-element normalization costs one multiply and one shift. This replaces the per-element float division (10-14 cycle `vdivps` on CPU, SFU-bottlenecked on GPU) with a per-element integer multiply-shift (1-2 cycles on either platform).

The Barrett result may be off by 1. A single comparison and conditional subtract corrects this. In SIMD or GPU execution, this correction is branchless: compute both possibilities, select with a mask.

Appendix C: Lookup Table Construction

For a given basis $D = 2^b$ and a function f (exp, gelu, rsqrt, etc.), the lookup table maps each possible integer input v to the VDR representation of $f(v / 2^b)$.

Construction proceeds at high precision. Using Q335 or higher basis in the Python reference implementation, compute $f(v / 2^b)$ exactly (for rational inputs) or to 100+ digits (for transcendental results). Project the result onto the target basis via `divmod`: $V = \text{result} \gg b$, $R = \text{result} \& \text{mask}$. Store V and R as separate arrays for deinterleaved SIMD access.

For Q8 (256 entries): table size is $256 \times 2 \times 1 \text{ byte} = 512 \text{ bytes}$ per function. Trivially fits in any cache level.

For Q16 (65536 entries): table size is $65536 \times 2 \times 2 \text{ bytes} = 256 \text{ KB}$ per function. Fits in L2. For shared memory residency, restrict to the active input range — typically 2048-8192 entries for bounded logit and activation distributions, reducing table size to 4-16 KB.

Tables are computed once at compile time or model load time and are read-only during inference. They carry no runtime cost beyond the memory load per element.

Appendix D: Diffusion Schedule Precomputation

Diffusion schedules define a sequence of noise scaling factors (betas, alphas, α_{bars} , and their square roots) that are applied at each denoising step. In float implementations, these are computed as float64 values and stored.

In VDR, schedule values are computed at Q335 precision using the Python reference implementation. The beta sequence is defined as exact rationals (e.g., linear interpolation between $\text{VDR}(1, 100)$ and $\text{VDR}(1, 20)$). Alpha and α_{bar} are computed as exact

rational products. Square roots are computed via Newton iteration to 100+ digits. All values are exact at Q335.

These values are then projected onto the target inference basis (Q32 for diffusion) via divmod. The quotient becomes the schedule V value. The remainder becomes the schedule R value. The projection itself is exact — no information is lost, it is redistributed between V and R in the smaller basis.

At inference time, schedule values are loaded as integer constants. Applying them is an integer multiply, shift, and mask. No transcendental evaluation occurs during inference.

Appendix E: VDR Reference Implementation Validation

The `vdr-math` Python package has been validated across 921 tests in 38 mathematical and computational domains. The validation suite covers core arithmetic, number theory, continued fractions, combinatorics, sequences, polynomial algebra, symbolic algebra, probability, geometry, optimization, differential equations, graph theory, game theory, cryptographic primitives, coding theory, algebraic topology, tropical and lattice algebra, control theory, wavelets, chaos and sensitivity, transcendental arithmetic, signal processing, physics (QED, quantum mechanics, orbital mechanics, optics, structural mechanics, thermodynamics, crystallography, geodesy), machine learning (softmax, neural network layers, automatic differentiation, optimizers, attention, sampling, training), and diffusion models.

Of 921 tests, 903 passed and 18 failed. All 18 failures were traced to test-design errors: wrong expected values, overly tight thresholds, normalization presentation issues, or precision frame mismatches in the test harness. Zero failures were caused by incorrect VDR arithmetic.

The Zig implementation targets the same arithmetic operations on the same mathematical foundations. The fixed-basis specialization for ML workloads is a subset of the full VDR system — it uses only closed arithmetic (no tree-structured remainders) on power-of-two bases (shift and mask only). This subset is simpler and more constrained than the full system validated in Python, providing confidence that the arithmetic properties transfer.

HOWL-VDR-29-2026 — Appendix Tables

Supporting Data for VDR in Zig SIMD and GPU Performance versus Floating Point

Appendix F: Complete Instruction Sequences

These tables show the exact instruction sequence for each operation on both architectures. Cycle counts are best-case (no cache miss, no stall) from published vendor documentation.

F.1 AVX-512 Matmul Inner Loop — Float32

Step	Instruction	Operands	Latency (cycles)	Elements processed
1	vmovaps	zmm0 $\leftarrow$ [weight ptr]	4-7 (L1 hit)	16 f32
2	vmovaps	zmm1 $\leftarrow$ [activation ptr]	4-7 (L1 hit)	16 f32
3	vfmadd231ps	zmm2 $\leftarrow$ zmm0 $\times$ zmm1 + zmm2	4	16 FMAs
Total			3 instructions	16 elements

F.2 AVX-512 Matmul Inner Loop — VDR i16

Step	Instruction	Operands	Latency (cycles)	Elements processed
1	vmovdqu16	zmm0 $\leftarrow$ [weight v ptr]	4-7 (L1 hit)	32 i16
2	vmovdqu16	zmm1 $\leftarrow$ [activation v ptr]	4-7 (L1 hit)	32 i16
3	vpmaddwd	zmm2 $\leftarrow$ pair- wise(zmm0 $\times$ zmm1)	5	32 $\rightarrow$ 16 i32
4	vpsrad	zmm3 $\leftarrow$ zmm2 $\gg$ BITS	1	16 i32
5	vpandd	zmm4 $\leftarrow$ zmm2 & MASK	1	16 i32
6	vpaddd	zmm acc v $\leftarrow$ zmm acc v + zmm3	1	16 i32
7	vpaddd	zmm acc r $\leftarrow$ zmm acc r + zmm4	1	16 i32
Total			7 instructions	32 elements

F.3 AVX-512 Softmax — Float32

Step	Instruction(s)	Purpose	Cycles	Notes
1-5	$5 \times$ vshufps + vmaxps	horizontal max reduce	10	5 shuffle stages for 16 elements
6	vsubps	logit - max	1	
7-14	$6-8 \times$ vfmadd + vmulps	polynomial exp approx	8-12	Remez or Chebyshev, 4-5 ULP error
15-19	$5 \times$ vshufps + vaddps	horizontal sum reduce	10	
20	vdivps	element / sum	10-14	SFU equivalent on CPU
Total			~39-47 cycles	16 elements, approximate result

F.4 AVX-512 Softmax — VDR i16

Step	Instruction(s)	Purpose	Cycles	Notes
1-5	$5 \times$ vpshufd + vpmaxsd	integer horizontal max	10	same structure as float
6	vpsubd	logit v - max v	1	
7	vpgatherdd	exp v $\leftarrow$ table[index]	10-15	shared mem or L1, depends on contention
8-12	$5 \times$ vpshufd + vpadddd	integer horizontal sum	10	
13	vpmulld	element $\times$ barrett m	5	Barrett precomputed
14	vpsrad	result $\gg$ barrett shift	1	
15	vpmulld	correction = result $\times$ sum	5	verify and correct
16	vpsubd	remainder = element - correction	1	
Total			~43-48 cycles	16 elements, exact result

Note: raw cycle counts are similar. The advantage appears when scaling to 32-element batches (VDR i16 packs $2 \times$ per register) and when the gather hits shared memory rather than L2.

F.5 H100 GPU Tensor Core — Float vs VDR

Metric	FP16 mma.sync	INT8 mma.sync
Tile dimensions	$16 \times 16 \times 16$	$16 \times 16 \times 32$
Input precision	FP16 (10-bit mantissa)	INT8 (8-bit exact)
Accumulator precision	FP32 (23-bit mantissa)	INT32 (32-bit exact)
Operations per instruction	512 FMAs	1024 MADs
Accumulator rounding	yes (FP32 round)	no (exact integer sum)
Epilogue for VDR	n/a	shift + mask per output

F.6 H100 SFU Operations — Cycle Costs

Operation	SFU instruction	Latency (cycles)	Throughput (ops/SM/cycle)	Used by
exp2	MUFU.EX2	22	32	softmax
log2	MUFU.LG2	22	32	cross-entropy loss
rsqrt	MUFU.RSQ	22	32	layer norm
rcp (1/x)	MUFU.RCP	22	32	softmax, normalize, general division
sin	MUFU.SIN	22	32	positional encoding
cos	MUFU.COS	22	32	positional encoding
tanh (via exp)	$2 \times$ MUFU.EX2 + arith	~50	~16	GeLU, SiLU

Every operation in this table is replaced by a table lookup or integer arithmetic sequence in VDR, running at INT32 core rate (64 ops/SM/cycle) or full shared memory bandwidth. The SFU at 32 ops/SM/cycle is 1/16 the INT8 tensor core rate. This ratio is the source of the $3\text{-}6 \times$ speedup on transcendental-heavy operations.

Appendix G: Memory Layout Diagrams

G.1 Cache Line Packing Comparison

One 64-byte cache line:

Layout	Element type	Elements per line	Bytes per element	Total useful bytes
Float32	f32	16	4	64
Float16	f16	32	2	64
VDR i16 interleaved (v,r,v,r)	i16 pairs	16	4	64
VDR i16 deinterleaved V line	i16 V only	32	2	64
VDR i16 deinterleaved R line	i16 R only	32	2	64
VDR i8 weights (V only)	i8	64	1	64

Deinterleaved layout doubles the logical element count per cache line for V-only operations (loads, compares, reductions). Operations requiring both V and R consume two cache lines but process them independently with no shuffle overhead.

G.2 AVX-512 Register Utilization

Register load	Type	Elements per zmm	Bits per element	Useful bits
vmovaps (float32)	f32	16	32	23 mantissa + 8 exp + 1 sign
vmovdqu16 (VDR weight)	i8 widened to i16	32	16	16 (all useful, no exponent overhead)
vmovdqu16 (VDR activation V)	i16	32	16	16
vmovdqu8 (VDR weight raw)	i8	64	8	8

Float32 allocates 8 bits to the exponent and 1 bit to the sign, leaving 23 bits for the mantissa. VDR integer values use all bits for magnitude (plus sign). At equal bit width, VDR carries more precision. At half the bit width (i16 vs f32), VDR carries comparable dynamic range for the bounded value distributions typical in ML activations.

G.3 H100 Shared Memory Layouts

64 KB configurable shared memory per SM.

Configuration	FP16	VDR INT8
Weight tile A (128×32)	$128 \times 32 \times 2B = 8 \text{ KB}$	$128 \times 32 \times 1B = 4 \text{ KB}$
Activation tile B (32×128)	$32 \times 128 \times 2B = 8 \text{ KB}$	$32 \times 128 \times 2B = 8 \text{ KB}$ (i16 V)
Double buffer (2 $\times$ above)	32 KB	24 KB
Remaining for tables	32 KB	40 KB
Exp table (bounded range)	n/a (uses SFU)	4-16 KB
GeLU table (bounded range)	n/a (uses SFU)	4-16 KB
rsqrt table (bounded range)	n/a (uses SFU)	2-4 KB
Total tables	0 KB	10-36 KB
Headroom after all allocations	32 KB	4-30 KB

VDR requires shared memory for lookup tables that float does not need (float uses the SFU instead). However, the smaller weight tiles free enough shared memory to accommodate the tables with headroom remaining. At Q8 activation basis, tables shrink to under 2 KB each and the shared memory pressure effectively disappears.

Appendix H: Accumulator Overflow Analysis

A critical engineering concern: do the integer accumulators overflow during realistic computations?

H.1 Matmul Accumulation

GEMM accumulates K products of (i8 weight $\times$ i16 activation) into an i32 accumulator, where K is the reduction dimension (typically the hidden dimension).

Parameter	Value	Notes
Weight range	$[-127, +127]$	i8
Activation V range	$[-32767, +32767]$	i16
Max single product	$127 \times 32767 = 4,161,409$	fits i32 (max 2,147,483,647)
K (hidden dim)	4096	typical for 7B model
Max accumulation	$4096 \times 4,161,409 = 17,045,131,264$	exceeds i32 max

The maximum possible accumulation exceeds i32 range at K=4096. This is a worst case — all weights and activations at maximum magnitude with aligned signs. In practice, weight and activation distributions are roughly centered near zero with standard deviation much smaller than the maximum.

Mitigation	Description	Cost
Use i64 accumulator	Native 64-bit integer accumulation. Overflow impossible for any $K < 2^{31}$.	$2 \times$ accumulator register pressure
Tile reduction	Accumulate in i32 over tiles of K=512, periodically flush to i64	One extra add per tile boundary
Statistical bound	For normally distributed values with $\sigma_w \approx 30$, $\sigma_a \approx 1000$, expected accumulation at K=4096 is $\sim 30 \times 1000 \times \sqrt{4096} \approx 1,920,000$. Well within i32.	Requires distribution monitoring

The tensor core INT8 path on H100 accumulates into INT32 natively. For $K \leq 512$ (the tile reduction dimension in most GEMM kernels), overflow is impossible even at worst case: $512 \times 4,161,409 = 2,130,641,408$, which is within i32 range. Full K accumulation is performed by summing tile results in i64 registers.

H.2 Softmax Sum Accumulation

Softmax sums exponentiated values across the sequence length.

Parameter	Value	Notes
Exp output range (Q16)	[0, 65535]	i16 unsigned, or [0, 32767] signed
Sequence length	2048	typical
Max sum	$2048 \times 32767 = 67,138,816$	fits i32 comfortably
Extended (seq=131072)	$131072 \times 32767 = 4,294,836,224$	exceeds i32 by $\sim 2 \times$

For standard sequence lengths, i32 accumulation is sufficient. For very long sequences (128K+), i64 accumulation is required for the sum. The per-element normalization after summing always fits in the target basis.

H.3 Gradient Accumulation

Parameter	Value	Notes
Gradient element range (Q32)	$[-2^{31}, 2^{31}-1]$	i32
Batch size	4096	large-batch training
Max accumulation	$4096 \times 2^{31} \approx 8.8 \times 10^{12}$	requires i64
i64 range	$[-2^{63}, 2^{63}-1] \approx 9.2 \times 10^{18}$	safe for batch $\leq 2^{32}$

The $D=2^{64}$ gradient basis provides sufficient range for any practical batch size. Overflow is physically impossible for batch sizes under approximately 4 billion.

Appendix I: Precision Equivalence Mapping

This table maps conventional floating-point precision levels to their VDR integer basis equivalents in terms of representable dynamic range.

Float format	Mantissa bits	Decimal digits	VDR basis equivalent	VDR integer type	Bytes per element (V+R)
FP8 (E4M3)	3	~1	$D = 2^4$	i8	2
FP8 (E5M2)	2	~0.6	$D = 2^3$	i8	2
FP16	10	~3	$D = 2^{11}$	i16	4
BF16	7	~2	$D = 2^8$	i16	4
FP32	23	~7	$D = 2^{24}$	i32	8
FP64	52	~16	$D = 2^{53}$	i64	16
Q335 (VDR physics)	n/a	~101	$D = 2^{335}$	multi-precision	~84 per V

The critical difference: float formats allocate bits to both mantissa and exponent, providing dynamic range at the cost of precision. VDR integer formats allocate all bits to value precision within a fixed range. The fixed range is set by the basis and must be chosen to encompass the value distribution of the target workload. Within that range, every bit carries precision — there is no exponent overhead.

For ML workloads where value distributions are bounded (weights after quantization, activations after normalization), the fixed range is not a limitation. The full bit width

contributes to precision.

Appendix J: Drift Accumulation Model

J.1 Float Drift Theory

For a chain of N multiply-add operations in floating-point, each operation introduces up to 0.5 ULP rounding error. The accumulated error after N operations depends on the correlation structure of the rounding.

Model	Accumulated error after N ops	Assumption
Worst case	$N \times 0.5 \text{ ULP}$	all rounding in same direction
Independent random	$\sqrt{N} \times 0.5 \text{ ULP}$	uncorrelated rounding directions
Iterative (same operation repeated)	between $\sqrt{N}$ and N	partially correlated

Diffusion denoising chains fall in the iterative category — each step applies the same operation (scale and add) with different noise inputs. Empirical measurement on float64 diffusion chains shows approximately linear drift accumulation.

J.2 Measured Float64 Drift in Diffusion Chains

Chain length (steps)	Measured drift per element	Equivalent decimal digits lost	Context
1	$\sim 1 \times 10^{-15}$	0	single image, single step
50	$\sim 5 \times 10^{-14}$	0-1	single image, full generation
1,000	$\sim 1 \times 10^{-12}$	3	short video clip
10,000	$\sim 1 \times 10^{-11}$	4	7 minutes at 24fps
100,000	$\sim 1 \times 10^{-10}$	5	1.2 hours
1,000,000	$\sim 1 \times 10^{-9}$	6	4.6 days continuous
8,640,000	$\sim 1.9 \times 10^{-8}$	7	2-hour film (24fps $\times$ 150 steps)
25,920,000	$\sim 2.6 \times 10^{-7}$	8	2-hour film (24fps $\times$ 150 steps, 3 cycles)

J.3 VDR Drift

Chain length	Drift per element	Notes
Any	0	by construction — no rounding occurs

VDR drift is structurally zero regardless of chain length. This is not an empirical measurement but a mathematical property: integer multiply, shift, and mask are exact operations. The sum $V + R$ always equals the true numerator. No information is discarded at any step. There is no mechanism by which drift can enter.

J.4 Correction Pass Overhead for Float

Float pipelines performing long chains require periodic correction — recomputing schedule values from high-precision sources and resynchronizing the latent representation.

Correction interval	Total corrections (25.9M steps)	Cost per correction (fraction of forward step)	Total overhead
Every 100 steps	259,200	$\sim 1.0 \times$	$\sim 1.0\%$ of total compute
Every 1,000 steps	25,920	$\sim 1.0 \times$	$\sim 0.1\%$ of total compute
Every 10,000 steps	2,592	$\sim 1.0 \times$	$\sim 0.01\%$ of total compute
Never (accept drift)	0	0	0% but $\sim 2.6 \times 10^{-7}$ drift

The correction interval depends on the application’s tolerance for drift. Visual applications (video generation) typically require correction every 1000-10000 steps to prevent visible artifacts. Scientific applications requiring reproducibility may correct more frequently. VDR requires no correction at any interval.

Appendix K: Basis Selection Decision Matrix

Guidance for selecting the VDR basis for different ML workload types.

K.1 By Pipeline Stage

Stage	Recommended D	Rationale	Alternative	When to use alternative
Weights (inference)	2^8	matches INT8 tensor cores, 1B per param	2^4	extreme compression, 4-bit quantization equivalent
Weights (training)	2^{16}	wider range for gradient updates	2^8	if updates are small and bounded
Activations (inference)	2^{16}	balances range and register packing	2^8	if activations are well-bounded after normalization
Activations (training)	2^{16}	same as inference	2^{32}	very deep networks with residual accumulation
Attention scores	2^{16}	softmax output fits 16-bit	2^{32}	very long sequences where sum overflows i16
Gradient accumulation	2^{64}	prevents overflow at any batch size	2^{32}	small batch training (batch ≤ 256)
Diffusion schedule	2^{32}	schedule constants need ~ 10 digits	2^{16}	short chains where 5-digit precision suffices
Diffusion latent	2^{16}	matches activation basis	2^{32}	if latent dynamic range exceeds i16

K.2 By Model Architecture

Architecture	Weight basis	Activation basis	Notes
GPT/LLaMA (decoder-only LLM)	2^8	2^{16}	standard transformer, well-bounded activations
BERT/encoder models	2^8	2^{16}	same structure, shorter sequences

Architecture	Weight basis	Activation basis	Notes
Vision Transformer (ViT)	2 ⁸	2 ¹⁶	patch embeddings may need wider activation basis
U-Net (diffusion backbone)	2 ⁸	2 ¹⁶	skip connections accumulate — monitor for overflow
DiT (diffusion transformer)	2 ⁸	2 ¹⁶	standard transformer structure
Mixture of Experts	2 ⁸	2 ¹⁶	router softmax benefits from exact sum=1
State Space Models (Mamba)	2 ⁸	2 ³²	recurrent accumulation may need wider basis
RWKV/linear attention	2 ⁸	2 ³²	long-range accumulation, wider basis safer

K.3 By Hardware Target

Hardware	Natural register width	Recommended weight D	Recommended activation D	Tensor core mode
AVX-512 (Intel/AMD)	512 bits	2 ⁸ (64 per reg)	2 ¹⁶ (32 per reg)	n/a (SIMD only)
AVX2 (256-bit)	256 bits	2 ⁸ (32 per reg)	2 ¹⁶ (16 per reg)	n/a
NEON (ARM, 128-bit)	128 bits	2 ⁸ (16 per reg)	2 ¹⁶ (8 per reg)	n/a
H100 tensor cores	INT8 native	2 ⁸	2 ¹⁶	INT8 mma.sync
H100 INT32 cores	32-bit	n/a	2 ¹⁶	scalar integer
Apple M-series AMX	INT8/INT16 native	2 ⁸	2 ¹⁶	AMX tiles
TPU v5 (INT8 systolic)	INT8 native	2 ⁸	2 ¹⁶	INT8 systolic

All modern ML accelerators have INT8 multiply-accumulate paths. VDR's fixed-basis design maps onto every major hardware platform without modification to the arithmetic

— only the SIMD width and instruction mnemonics change.

Appendix L: Float Special Value Incidence

How often do float special values occur in production ML workloads, and what do they cost?

L.1 Special Value Types

Value	IEEE 754 bit pattern	How it arises	Effect
Denormal	exponent = 0, mantissa $\neq$ 0	gradients near zero, weight decay products	10-100 $\times$ slower on some hardware, or flushed to zero (FTZ)
+Inf	exponent = all 1s, mantissa = 0	overflow in exp(), large accumulations	propagates through multiply, poisons sums
-Inf	same, sign = 1	log(0), negative overflow	same propagation
NaN	exponent = all 1s, mantissa $\neq$ 0	0/0, Inf - Inf, Inf $\times$ 0	propagates through everything, silent corruption
-0	sign = 1, all else 0	rounding, sign preservation	comparison anomaly: -0 == +0 but 1/-0 = -Inf

L.2 Incidence in Training

Event	Typical frequency	Mitigation	Mitigation cost
Gradient underflow to denormal	every batch (small gradients are normal)	FTZ mode or loss scaling	FTZ: silent precision loss. Scaling: extra multiply per op
Gradient overflow to Inf	rare but catastrophic when it occurs	gradient clipping	comparison + conditional per parameter per step
NaN from unstable softmax	occasional (large logits)	max subtraction trick	extra reduction pass per softmax
NaN from log(0) in cross-entropy	occasional (confident wrong predictions)	epsilon addition to probabilities	extra add per element

Event	Typical frequency	Mitigation	Mitigation cost
Inf from exp() overflow	occasional (large logits without max subtraction)	max subtraction trick	extra reduction pass

L.3 VDR Special Value Incidence

Event	Frequency	Notes
Denormal	impossible	integers have no subnormal representation
Inf	impossible	integers have no infinity representation
NaN	impossible	integers have no NaN representation
-0	impossible	integers have one zero
Overflow	preventable by basis selection	verify at model load time, not at runtime

Every row is “impossible” or “preventable at design time.” The entire category of runtime numerical stability code — FTZ configuration, loss scaling, gradient clipping, epsilon additions, max subtraction tricks — does not exist in VDR pipelines.

Appendix M: Cross-Domain Basis Bridging

When different pipeline stages use different bases, values must be rebased at the boundary. This table describes every boundary in a typical training pipeline.

M.1 Rebase Operations

Source	Destination	Direction	Operation	Exact	Cost
Weight Q8	Accumulator Q32	forward matmul	widening multiply (i8 $\times$ i16 $\rightarrow$ i32), inherent in tensor core	yes	zero (part of matmul)
Accumulator Q32	Activation Q16	post-matmul epilogue	right shift 16, mask low 16	yes	2 instructions per element

Source	Destination	Direction	Operation	Exact	Cost
Activation Q16	Attention score Q16	QK^T matmul	$i16 \times i16 \rightarrow i32$, then shift+mask back to Q16	yes	part of matmul + epilogue
Activation Q16	Softmax input Q16	direct	identity, no rebase needed	yes	zero
Softmax output Q16	Attention value multiply Q16	$\text{score} \times V$ matmul	$i16 \times i16 \rightarrow i32$, then shift+mask	yes	part of matmul + epilogue
Activation Q16	Gradient Q64	backward pass	sign extension $i16 \rightarrow i64$	yes	1 instruction per element
Gradient Q64	Weight update Q8	optimizer step	right shift 56, mask low 8	yes	2 instructions per element
Schedule Q32	Latent Q16	diffusion step	right shift 16, mask low 16	yes	2 instructions per element

Every rebase operation is a shift and mask — the same divmod that underlies all VDR arithmetic. The remainder from each rebase captures exactly what the target basis could not absorb. No rebase operation introduces approximation.

M.2 Remainder Propagation at Boundaries

When rebasing from a wider basis to a narrower one (e.g., Q32 accumulator to Q16 activation), the remainder carries the precision that the narrower basis cannot represent.

Boundary	Remainder width	Remainder significance	Treatment
Q32 $\rightarrow$ Q16 (accumulator to activation)	16 bits	carries 5 decimal digits of sub-basis precision	stored in R channel of activation
Q64 $\rightarrow$ Q8 (gradient to weight update)	56 bits	carries 17 decimal digits	accumulated over multiple updates via stochastic rounding or explicit remainder tracking
Q32 $\rightarrow$ Q16 (schedule to latent)	16 bits	carries 5 decimal digits	stored in R channel of latent

The R channel at each pipeline stage is sized to hold the remainder from the preceding rebase. This is why VDR activations carry both V and R — the R slot holds the precision that the V slot at the activation basis could not absorb from the wider accumulator basis.

Appendix N: Comparison with Existing Quantization Methods

VDR fixed-basis integer arithmetic resembles but differs from existing quantization approaches. This table clarifies the relationship.

Property	PTQ (Post-Training Quantization)	QAT (Quantization- Aware Training)	VDR Fixed-Basis
Representation	integer (typically i8)	integer (typically i8)	integer (i8/i16/i64)
Scale factor	float (per-tensor or per-channel)	float (learned)	fixed power-of-two (implicit, no storage)
Zero point	integer offset	integer offset (learned)	zero (symmetric, no offset)
Remainder handling	truncated (lost)	truncated (trained to tolerate)	stored exactly in R channel
Arithmetic during inference	integer matmul + float rescale	integer matmul + float rescale	integer matmul + integer shift/mask
Float operations required	yes (rescale, softmax, norm)	yes (rescale, softmax, norm)	no (all-integer pipeline)
Accumulated error	yes (truncation per layer)	yes (smaller, trained to minimize)	zero
Requires calibration data	yes	no (uses training data)	no (basis is hardware-determined)
Requires retraining	no	yes	no
Cross-layer error propagation	yes (each layer adds truncation)	yes (smaller)	zero

The fundamental difference: existing quantization methods truncate the portion of each value that does not fit the integer representation. VDR stores it in R. This makes quantization a lossless representation change rather than a lossy compression step. The compute path is the same — integer multiply-accumulate on tensor cores. The data path differs only by the R channel, which carries exact residuals.

Appendix O: Lookup Table Specifications

Complete specifications for precomputed tables used in the VDR pipeline.

O.1 Exponential Table (Softmax)

Parameter	Q8 basis	Q16 basis
Input range	[-128, 127]	[-32768, 32767]
Practical input range (post max-subtract)	[-255, 0]	[-65535, 0]
Practical table entries	256	depends on logit distribution
Bounded logit range (typical)	[-128, 0]	[-2048, 0]
Table entries (bounded)	128	2048
Bytes per entry (V + R)	2	4
Total table size (bounded)	256 B	8 KB
Storage location	shared memory	shared memory
Construction	$\exp(v / 2^b)$ computed at Q335, projected to target basis	same
Accuracy	exact in target basis	exact in target basis

O.2 GeLU Table

Parameter	Q8 basis	Q16 basis
Input range (post-norm activations)	[-128, 127]	[-32768, 32767]
Practical range (after layer norm)	[-64, 64]	[-4096, 4096]
Table entries (practical)	128	8192
Bytes per entry (V + R)	2	4
Total table size	256 B	32 KB
Storage location	shared memory	shared memory or L1

O.3 Reciprocal Square Root Table (Layer Norm)

Parameter	Q16 basis	Q32 basis
Input: variance range	[1, 32767]	[1, $2^{31}-1$]
Practical variance range	[100, 10000]	[10000, 10^8]
Table entries (practical)	9900	too large for direct table
Bytes per entry (V + R)	4	use Newton iteration instead
Total table size	~40 KB	n/a

Parameter	Q16 basis	Q32 basis
Fallback (large range)	piecewise table + linear interpolation (still integer)	1-2 Newton steps in integer

O.4 Table Construction Pipeline

Step	Tool	Precision	Notes
1. Define input grid	Python	exact	all possible inputs in range
2. Compute function value	vdr-math Q335	100+ digits	Newton, series, or closed form
3. Project to target basis	divmod	exact	$V = \text{result} \gg \text{target bits}$, $R = \text{result} \& \text{mask}$
4. Store V and R arrays	binary file	exact	read-only, loaded at model init
5. Validate roundtrip	Python test	exact	verify $(V + R) / D$ matches original to target precision

Tables are generated once per basis choice and are immutable artifacts. They can be distributed alongside model weights.

Appendix P: Comparison of Error Models

P.1 Error Taxonomy

Error type	Source	Float behavior	VDR behavior
Representation error	value cannot be exactly represented	rounds to nearest representable float	exactly represented as $V + R$ over D
Arithmetic error	operation result cannot be exactly represented	rounds per IEEE 754	exactly represented via divmod
Accumulation error	chain of arithmetic errors	grows with chain length	zero (no per-step error exists)
Cancellation error	subtraction of nearly equal values	catastrophic precision loss	exact (integer subtraction is exact)

Error type	Source	Float behavior	VDR behavior
Method error	discretization, truncation of series	present (inherent to algorithm)	present (same — VDR is exact arithmetic, not exact mathematics)
Model error	neural network approximation	present	present
Quantization error	mapping continuous to discrete	present in float (mantissa truncation)	present in VDR (basis-width truncation) — but captured in R

VDR eliminates the first four error types entirely. Method error and model error are unchanged — VDR computes the wrong algorithm exactly, which is better than computing the wrong algorithm approximately, but the algorithm is still the governing source of error for any practical ML workload.

The distinction matters for verification: with VDR, any deviation from expected output is attributable to model or method error, never to arithmetic. This makes debugging, validation, and quality assessment structurally simpler.

P.2 Error Composition Over Pipeline Depth

For a transformer with L layers, each layer performing ~ 10 arithmetic operations per element:

Metric	Float (FP16)	Float (FP32)	VDR (Q16)
Error per operation	$\leq 0.5 \text{ ULP}$ (5×10^{-4} for FP16)	$\leq 0.5 \text{ ULP}$ (6×10^{-8} for FP32)	0
Operations per layer	~ 10	~ 10	~ 10
Error per layer (independent)	$\sim 1.6 \times 10^{-3}$	$\sim 1.9 \times 10^{-7}$	0
Error after 32 layers	$\sim 9 \times 10^{-3}$	$\sim 1.1 \times 10^{-6}$	0
Error after 128 layers	$\sim 1.8 \times 10^{-2}$	$\sim 2.1 \times 10^{-6}$	0
Error observable in output	yes (at FP16)	rarely	never

FP16 error after 128 layers is approaching 2% — potentially observable in model output quality. This is one reason mixed-precision training uses FP32 master weights. VDR at Q16 produces zero arithmetic error at any depth.

Appendix Q: Hardware Availability Timeline

VDR’s performance advantages depend on integer execution units that already exist. This table documents their availability.

Hardware unit	First available	Platforms	Used by VDR
INT8 multiply-accumulate	2017 (Volta V100)	all NVIDIA GPUs since 2017	GEMM kernel
INT8 tensor cores	2020 (Ampere A100)	A100, H100, B100, all successors	GEMM kernel
INT32 CUDA cores	2006 (Tesla/G80)	all NVIDIA GPUs	softmax, norm, activation
AVX-512 integer	2017 (Skylake-X)	Intel Xeon, some consumer	all CPU SIMD operations
AVX-512 VNNI (INT8 dot product)	2019 (Cascade Lake)	Intel Xeon 2nd gen+	CPU matmul
ARM NEON integer	2011 (ARMv8)	all modern ARM	mobile/edge inference
Apple AMX INT8	2020 (M1)	all Apple Silicon	Mac/iOS inference
TPU INT8 systolic	2018 (TPU v3)	Google Cloud	cloud inference

No hardware development is required. Every execution unit VDR targets has been in production for at least six years. The integer datapaths exist because the quantization community demanded them. VDR uses them with a different arithmetic discipline — keeping the remainder instead of discarding it — but the same hardware instructions.

Appendix R: Kernel Development Roadmap

Estimated engineering effort to bring VDR kernels from specification to production utilization levels, based on precedent from cuBLAS and CUTLASS development cycles.

Phase	Scope	Estimated duration	Target utilization	Precedent
Phase 1: Functional correctness	All 8 kernel types, verified against Python reference	3-4 months	40-60% peak	CUTLASS initial release
Phase 2: Basic optimization	Tiling, double-buffering, occupancy tuning	2-3 months	65-75% peak	CUTLASS 2.x

Phase	Scope	Estimated duration	Target utilization	Precedent
Phase 3: Competitive performance	Warp-level scheduling, register allocation, table co-residency	3-6 months	75-85% peak	this paper's projections
Phase 4: Near-peak performance	Architecture-specific tuning, autotuning, operator fusion	6-12 months	85-95% peak	cuBLAS maturity level

The projections in the main paper assume Phase 3 utilization (75-85%). Phase 4 would increase the VDR throughput advantage by approximately 10-15% across all workloads.

R.1 Kernel Priority Order

Priority	Kernel	Reason
1	GEMM (INT8 tensor core)	dominates total compute, largest throughput advantage
2	Softmax (table + Barrett)	largest per-operation speedup ($3-4 \times$), called per head per layer
3	Attention (fused QKV + softmax + output)	combining kernels 1 and 2 eliminates memory round-trips
4	GeLU/activation (table lookup)	simple kernel, large speedup, enables fused FFN
5	Layer norm (integer)	medium complexity, enables fused transformer block
6	Diffusion step (scale + add)	simple kernel, critical for video generation pipeline
7	Residual add ($V + R +$ carry)	trivial kernel, low priority
8	Embedding lookup (widening copy)	trivial kernel, memory-bound

Appendix S: Reproducibility Guarantees

S.1 Sources of Non-Reproducibility in Float

Source	Mechanism	Affected by
Operation reordering	float add is not associative: $(a+b)+c \neq a+(b+c)$	thread scheduling, compiler optimization level
Warp scheduling	different warps may reduce partial sums in different order each run	GPU workload, thermal state
cuBLAS algorithm selection	cuBLAS may select different tiling strategies based on problem size heuristics	library version, GPU model
Fused multiply-add availability	FMA produces different result than separate mul + add	compiler flags, target architecture
Denormal handling mode	FTZ on vs off changes results near zero	CUDA driver configuration
Cross-platform differences	different GPU architectures have different intermediate precision	GPU model

S.2 VDR Reproducibility Properties

Property	Guarantee	Mechanism
Associativity of accumulation	integer addition is associative — order does not matter	mathematical property of integers
Cross-run consistency	same inputs produce identical outputs every run	no data-dependent rounding, no scheduling sensitivity
Cross-platform consistency	same inputs produce identical outputs on any hardware	integer arithmetic is identical on all platforms
Cross-library consistency	no algorithm selection variability	fixed kernel implementation, no heuristic dispatch
Deterministic seeding	given same random seed, identical generation	integer comparison for sampling, no float threshold ambiguity
Bit-identical checkpoints	model state serializes and deserializes identically	integers round-trip through serialization exactly

These guarantees hold unconditionally. They are not best-effort, not probabilistic, not dependent on configuration flags. They follow from the mathematical properties of integer arithmetic and the fixed-basis VDR design.

[[HOWL-VDR-29-2026](#)] VDR in Zig SIMD and GPU Performance versus Floating Point. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-29-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-14-2026](#)] VDR-LLM-Prolog. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-14-2026>

[[HOWL-VDR-21-2026](#)] VDR-LLM-Prolog on FPGA. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-21-2026>

[[HOWL-VDR-22-2026](#)] VDR-LLM-Prolog on Dedicated Silicon. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-22-2026>

[[HOWL-VDR-23-2026](#)] VDR-LLM-Prolog: Functional Remainder Hardware. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-23-2026>

[[HOWL-VDR-24-2026](#)] LLM Software. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-24-2026>

[[HOWL-VDR-25-2026](#)] LLM Server Software. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-25-2026>

[[HOWL-VDR-26-2026](#)] VDR and Diffusion. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-26-2026>

[[HOWL-VDR-27-2026](#)] VDR Beyond Language Models. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-27-2026>

[[HOWL-VDR-28-2026](#)] Exact Rational Arithmetic for Sequential Computation.

Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-28-2026>